

Министерство науки и образования РФ
федеральное государственное автономное образовательное учреждение высшего образования
«Омский государственный технический университет»

Факультет (институт) Информационных технологий и компьютерных систем
Кафедра Информатики и вычислительной техники

Расчетно-графическая работа

по дисциплине Программирование
на тему Разработка программы «Расчет и построение графиков функций,
решение нелинейного уравнения и вычисление интеграла»

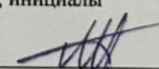
Студента (ки) Шмидта Антона Владиславовича
фамилия, имя, отчество полностью

Курс I Группа ИВТ-244

Направление (специальность) 09.03.01 –
Информатика и вычислительная техника
код, наименование

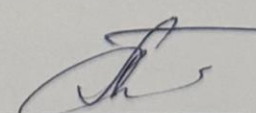
Руководитель к.т.н., доцент
ученая степень, звание

Шафеева О.П.
фамилия, инициалы

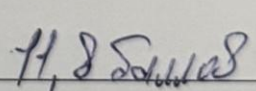
Выполнил (а) 02.06.2025 
дата, подпись студента (ки)

Работа защищен (а) с количеством
баллов

03 ИЮН 2025



дата, подпись руководителя


Шафеева О.П.

Омск 2025

Содержание

Министерство науки и образования РФ	1
Текстовые формулировки задач	3
Математическая формулировка задачи	4
Общая схема алгоритмов.....	6
Код	24
Разработка пользовательского интерфейса	40
Список литературы	45

Текстовые формулировки задач

Разработать схему алгоритма, написать и отладить следующие программы:

1. Анимационная заставка.
2. Вывод информации об авторе работы.
3. Расчёт и построение графиков двух функций (результаты расчётов должны храниться в виде массивов и распечатываться в виде таблицы), выделить наибольшее и наименьшее значения для каждой из функций.

Исходные данные для функций:

1) $f_1(x) = 5 - 3 \times \cos x$

2) $f_2(x) = \sqrt{1 + (\sin x)^2}$

3) Промежуток: $[0; 2\pi]$

4) Количество точек: 500

4. Нахождение корней уравнения $f(x) = 0$ на интервале $[a, b]$ с точностью $e = 0.001$ (интервал подобрать или рассчитать самостоятельно). При реализации можно использовать метод половинного деления (бисекции) или метод хорд.

Исходные данные для уравнения:

1) $f(x) = 0.5 + \cos x - 2 \times x \times \sin x$

2) Промежуток: $[0; 2]$

5. Вычисление значения определенного интеграла на интервале $[a, b]$ (a, b подобрать самостоятельно) численными методами прямоугольников и трапеций для следующего интеграла:

Исходные данные интеграла:

1) $\int_a^b e^{-x} \times \ln(x + 1) dx$

2) Промежуток: $[0; 2]$

3) Количество разбиений: 1000

Математическая формулировка задачи

Задание 1: Для нахождения данных функций заданная область их определения равномерно делится на n частей ($n = 20$). Длина полученных интервалов (шаг) находится по формуле $dx = \frac{|b-a|}{n-1}$. Затем вычисляются значения x в каждой точке, которые в цикле подставляются в функции. Результаты выводятся в виде таблицы. Далее находятся минимальные и максимальные значения функций. Вычисления производятся для функций: $f_1(x) = 5 - 3 \times \cos x$ и $f_2(x) = \sqrt{1 + (\sin x)^2}$.

Задание 2: Нахождение корня уравнения.

Метод половинного деления (бисекции) используется для приближённого решения уравнения $f(x) = 0$, если функция непрерывна на отрезке $[a, b]$ и $f(a)$ и $f(b)$ имеют разные знаки. На каждом шаге вычисляют середину отрезка $c = \frac{a+b}{2}$ и проверяют знак $f(c)$. В зависимости от него отрезок делят пополам, сохраняя тот, где функция меняет знак. Повторяя это, постепенно сужают отрезок до нужной точности.

Метод хорд основан на приближении графика функции прямой (хордой), проведённой через две точки. Новая точка вычисляется по формуле:

$$x_{n+1} = x_n - \frac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}$$

Этот метод сходится быстрее, чем бисекция, но может не сойтись, если начальные приближения выбраны неудачно.

Задание 3: Нахождение приближенного значения определенного интеграла.

Метод прямоугольников заключается в том, что отрезок интегрирования делится на равные части, и на каждом из полученных промежутков функция заменяется прямоугольником. Высоту прямоугольника определяют по значению функции в какой-либо одной точке промежутка (например, в левой или правой границе, или в середине). Затем находят площади всех прямоугольников и складывают их. Полученная сумма приближённо равна значению интеграла.

$$\int_a^b f(x)dx \approx h \times \sum_{i=0}^{n-1} f(x_i)$$

Метод трапеций тоже делит отрезок интегрирования на равные части, но вместо прямоугольников используется приближение кривой отрезками прямых, образующими трапеции. Площадь каждой трапеции вычисляется как среднее

арифметическое значений функции на концах промежутка, умноженное на ширину промежутка. Сумма площадей всех трапеций даёт приближённое значение интеграла.

$$\int_a^b f(x)dx \approx \frac{h}{2} \times \left[f(x_0) + 2 \times \sum_{i=0}^{n-1} f(x_i) + f(x_n) \right]$$

Общая схема алгоритмов

Расчетно-графическая работа состоит из следующих частей:

- Анимированная заставка;
- Об авторе;
- Вычисление значений двух функций с выводом результатов на экран в виде таблицы и выделением минимальных и максимальных значений;
- Построение графиков для функций;
- Решение уравнения двумя способами;
- Вычисление интеграла двумя способами;
- Выход.

Для выполнения этих задач было разработано меню, управление в котором осуществляется путем нажатия стрелок на клавиатуре. Для вывода нужной подпрограммы следует нажать на клавишу Enter.

На рисунке 1 представлена общая схема алгоритма работы меню. На клавиатуре с помощью стрелок выбирается пункт и нажимается Enter. Для возврата из подпрограммы обратно в меню нужно нажать клавишу Esc. Если в меню пользователь выбирает пункт Выход, то программа завершает свою работу. Программа предоставляет информацию об авторе, решает функции, определяет и выделяет цветом минимальные и максимальные значения функций, строит их графики на заданном отрезке, решает уравнение и определенный интеграл, а также выводит динамическую заставку.

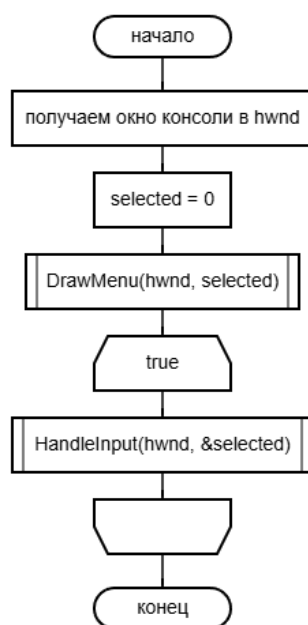


Рисунок 1 – схема алгоритма главной функции.

Детальные схемы алгоритмов

В программе были объявлены глобальные переменные в файле «functions.cpp»:

```
const int MENU_ITEMS = 7;
const char* menuItems[MENU_ITEMS] = {
    "1. Заставка",
    "2. Об авторе",
    "3. Таблица",
    "4. График",
    "5. Интеграл",
    "6. Уравнение",
    "7. Выход"
};
```

Так же был создан заголовочный файл «functions.h», который подключался к каждому файлу проекта с помощью «#include "functions.h"»:

```
#pragma once
#define _USE_MATH_DEFINES
#include <windows.h>
#include <string>
#include <cstring>
#include <cmath>
#include <iomanip>
#include <sstream>
#include <vector>

using namespace std;

extern const int MENU_ITEMS;
extern const char* menuItems[];

void DrawMenu(HWND hwnd, int selected);
void HandleInput(HWND hwnd, int* selected);
void ShowAbout(HWND hwnd);
void ShowTable(HWND hwnd);
void ShowGraph(HWND hwnd);
void ShowIntegral(HWND hwnd);
void ShowEquation(HWND hwnd);
void ShowIntro(HWND hwnd);
```

На рисунке 2 приведена схема отрисовки главного меню программы.

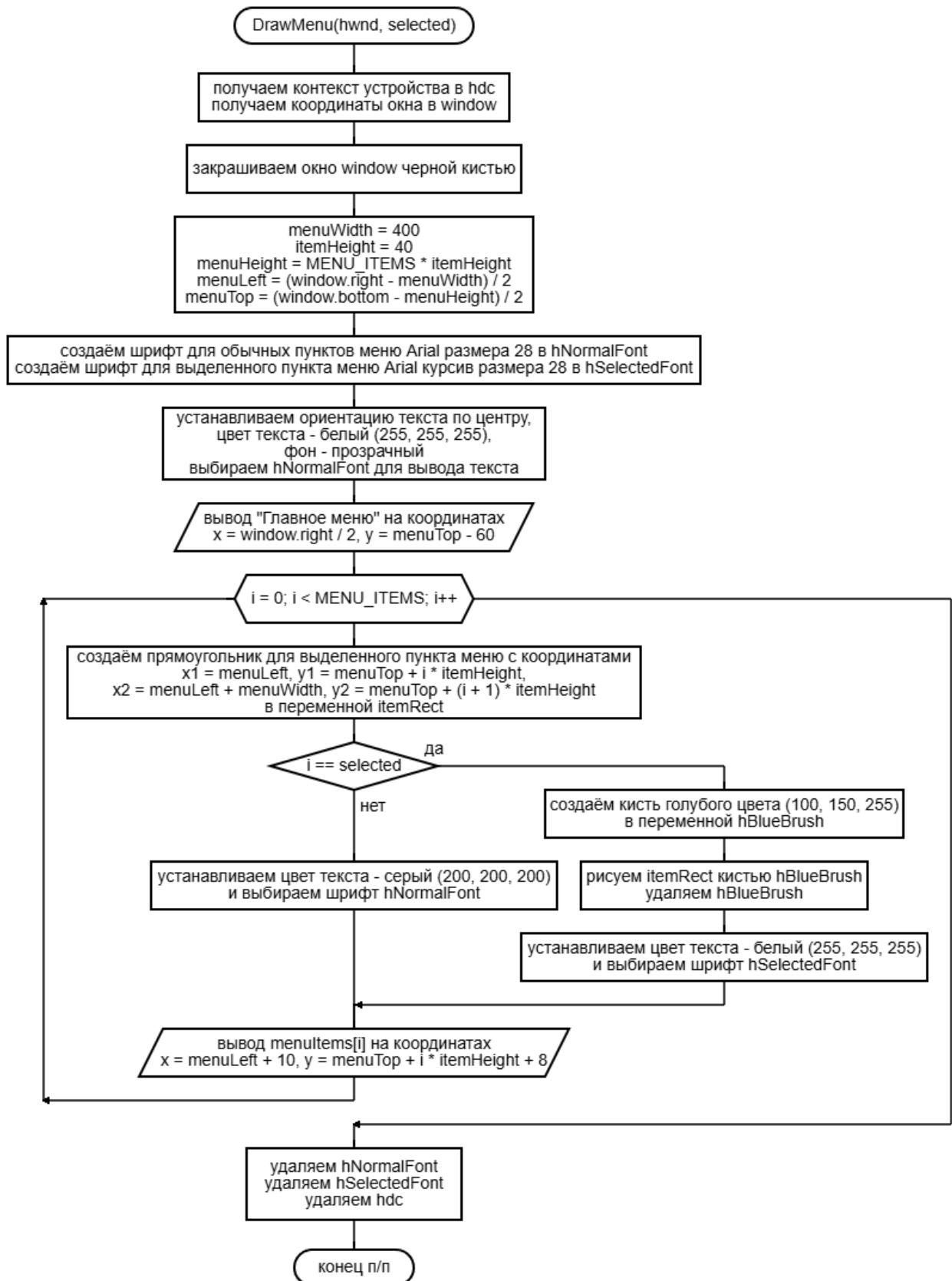


Рисунок 2 – схема алгоритма вывода главного меню.

На рисунке 3 показан алгоритм обработки ввода клавиш в главном меню программы.

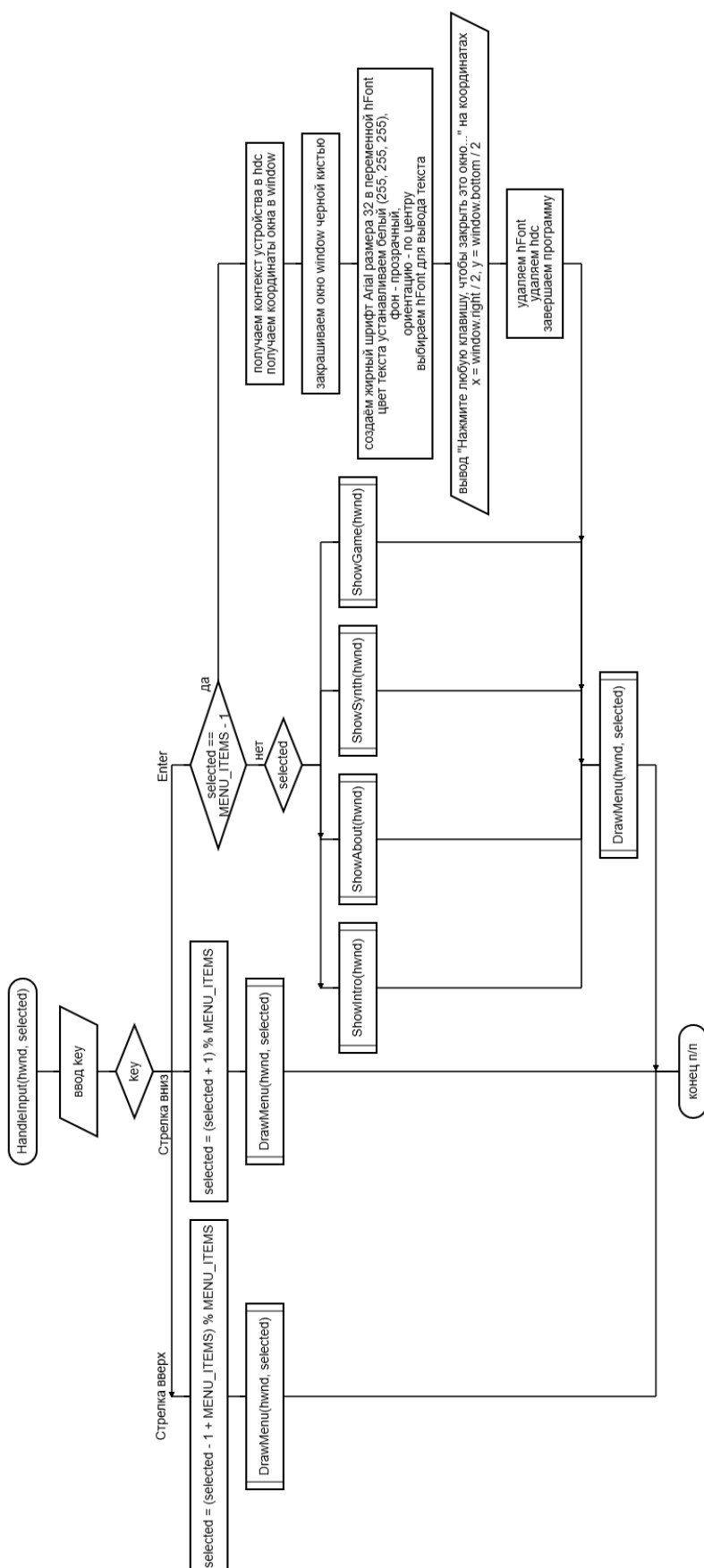


Рисунок 3 – схема алгоритма работы главного меню.

На рисунке 4 показана схема отрисовки динамической заставки программы.

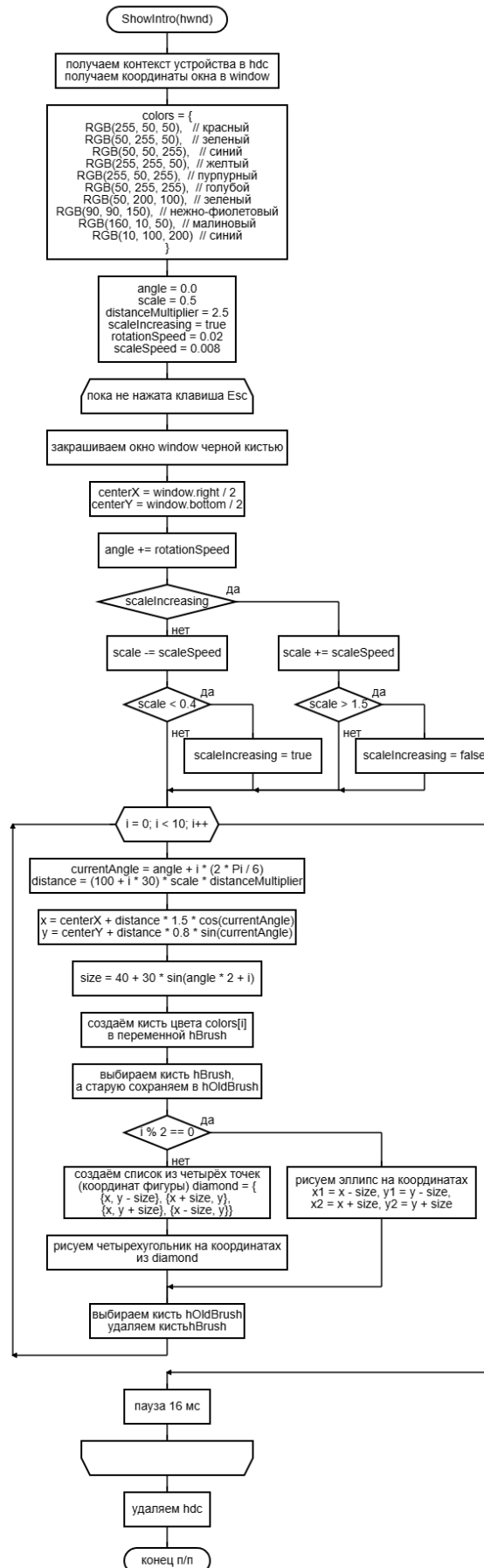


Рисунок 4 – схема алгоритма динамической заставки.

На рисунке 5 приведен алгоритм для вывода информации об авторе программы.

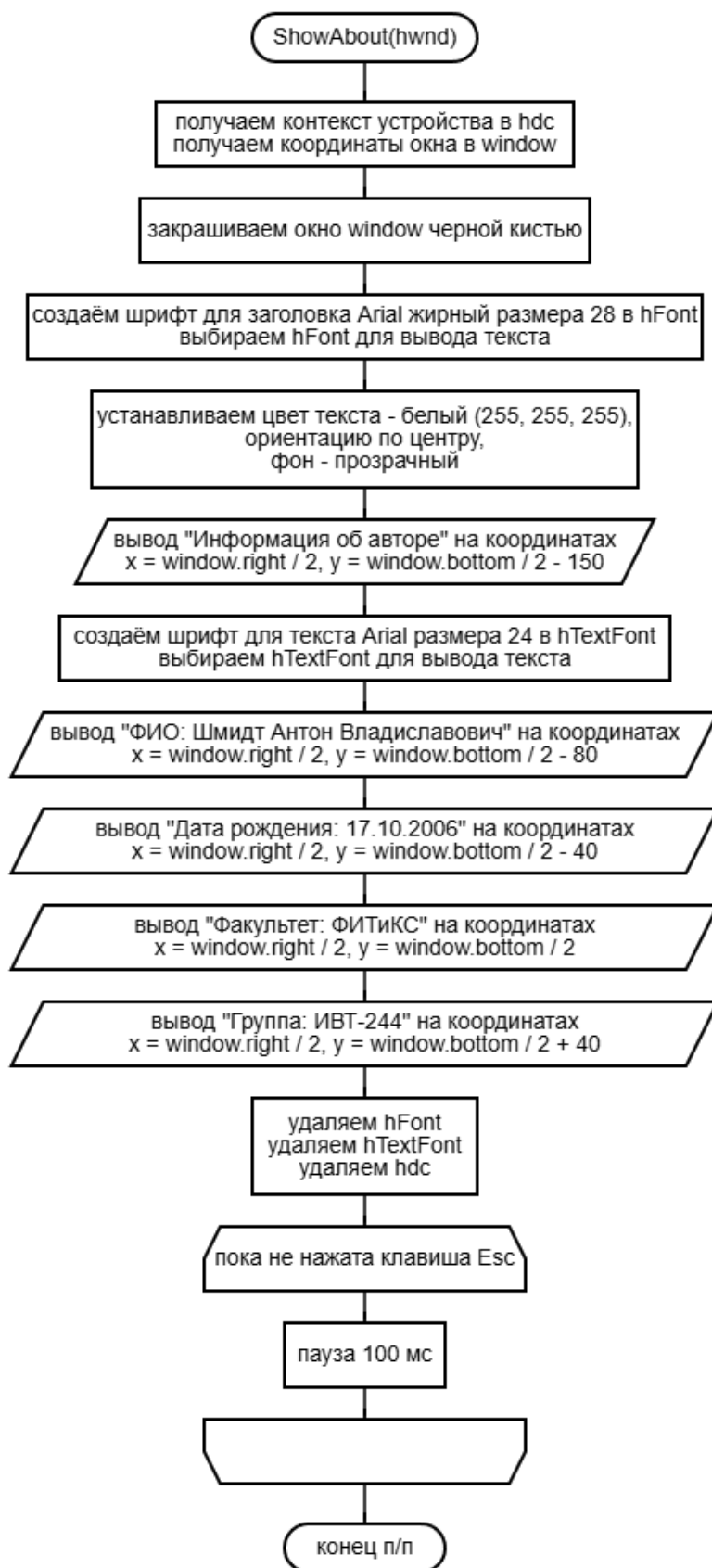


Рисунок 5 – схема алгоритма вывода информации об авторе.

На рисунке 6 показан алгоритм подсчёта значений функций на промежутке $[0; 2\pi]$ и вывода их на экран в виде таблицы.

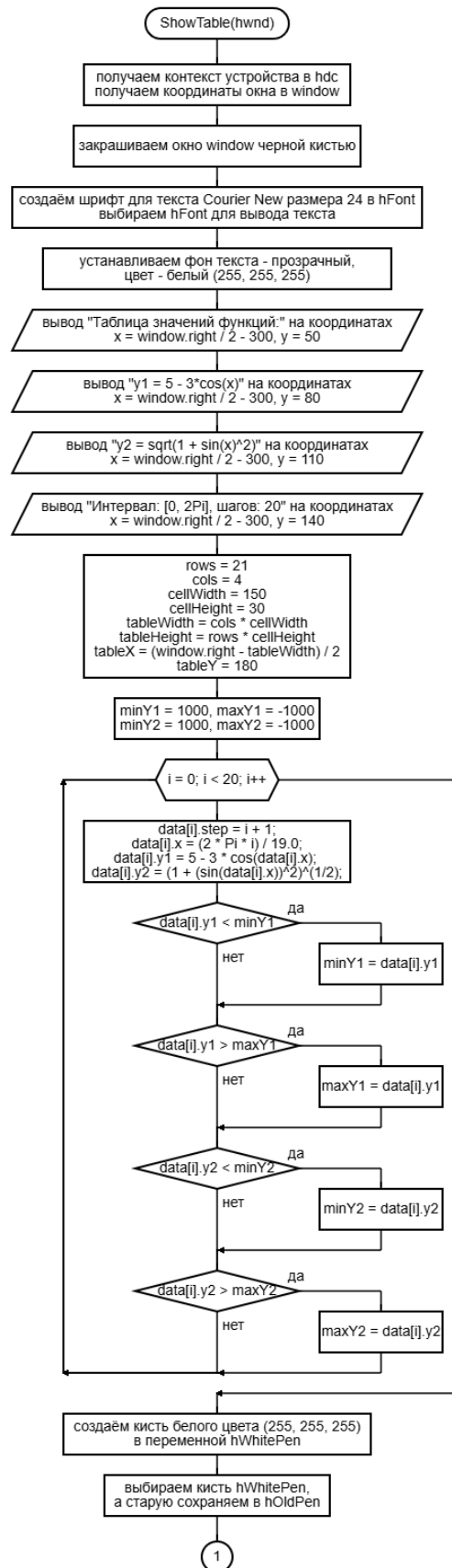
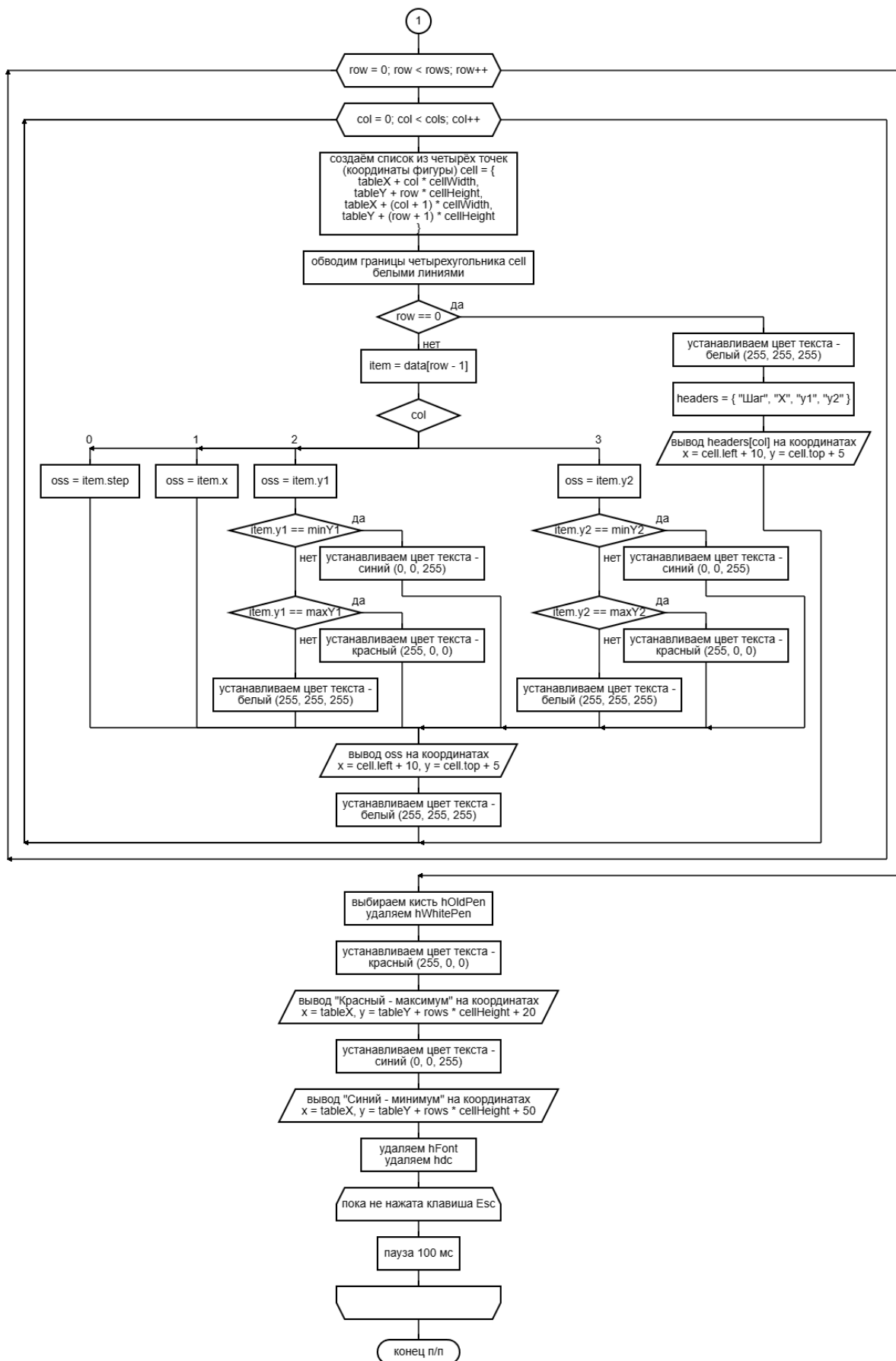


Рисунок 6 – схема алгоритма вывода таблицы с табуляцией функций.



Продолжение рисунка 6 – схема алгоритма вывода таблицы с табуляцией функций.

На рисунке 7 показан алгоритм отрисовки графиков функций.

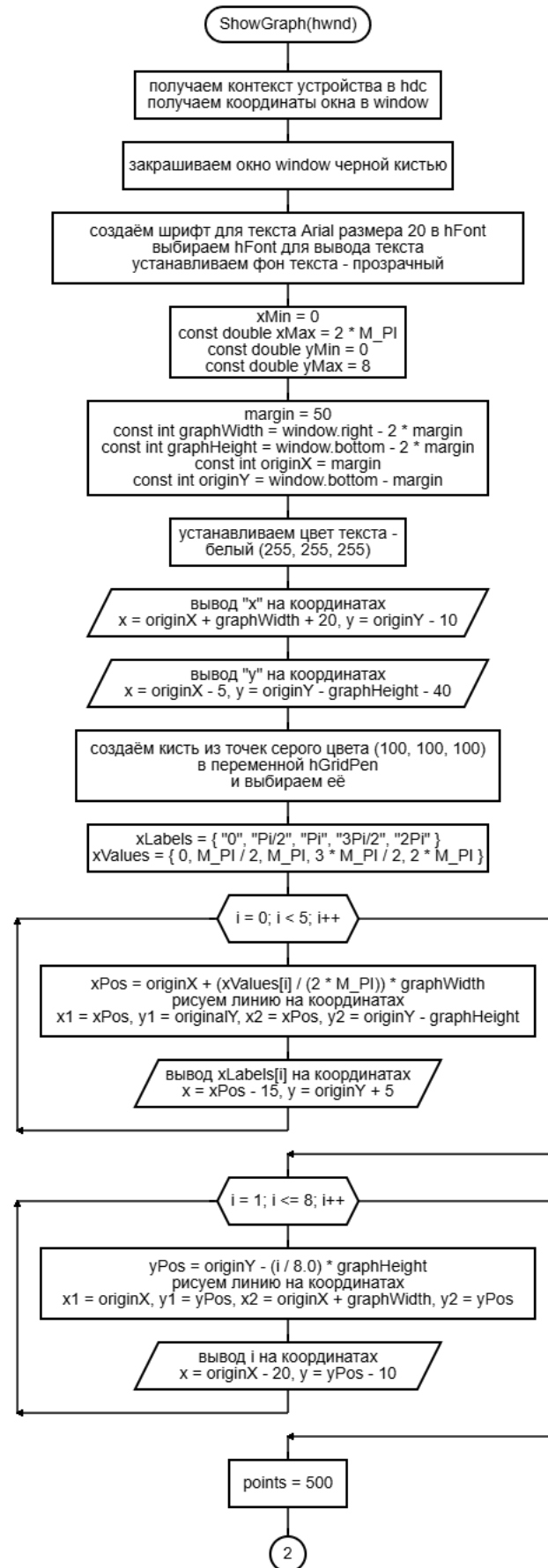
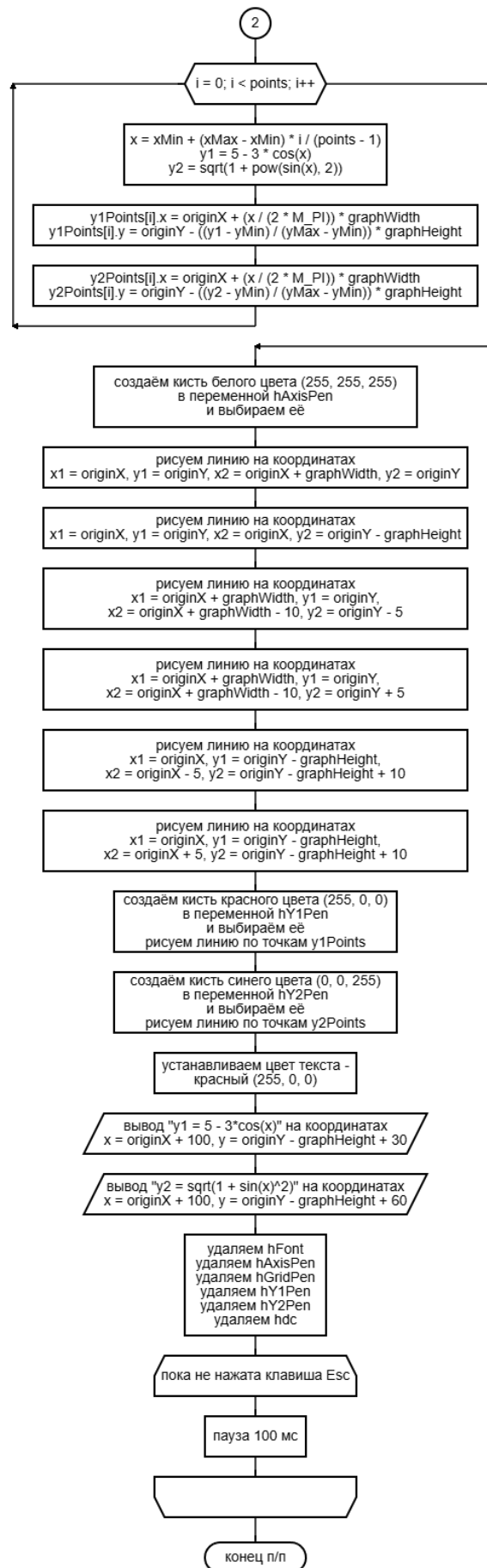


Рисунок 7 – схема алгоритма вывода графиков функций.



Продолжение рисунка 7 – схема алгоритма вывода графиков функций.

На рисунке 8 показан алгоритм подсчёта значения подынтегральной функции.

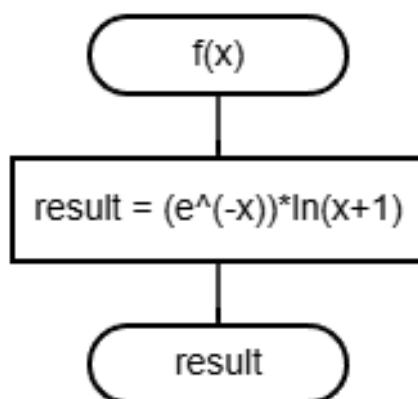


Рисунок 8 – схема алгоритма подсчёта значения функции из интеграла.

На рисунке 9 представлена схема подсчёта значения определенного интеграла методом прямоугольников.

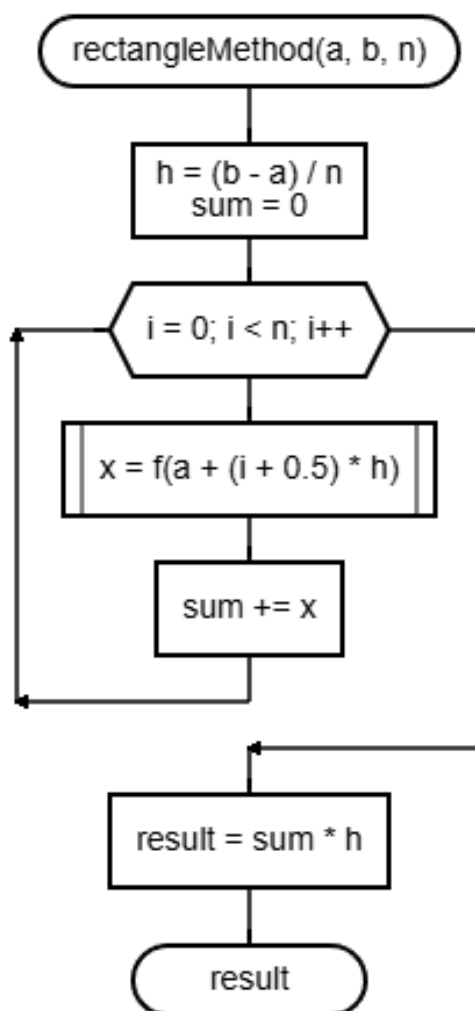


Рисунок 9 – схема алгоритма подсчёта интеграла методом прямоугольников.

На рисунке 10 показана схема подсчёта значения определённого интеграла методом трапеций.

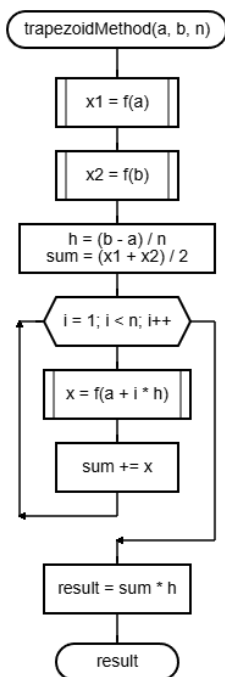


Рисунок 10 – схема алгоритма подсчёта интеграла методом трапеций.

На рисунке 11 показана схема вывода результатов подсчётов значений определенного интеграла методами прямоугольников и трапеций.

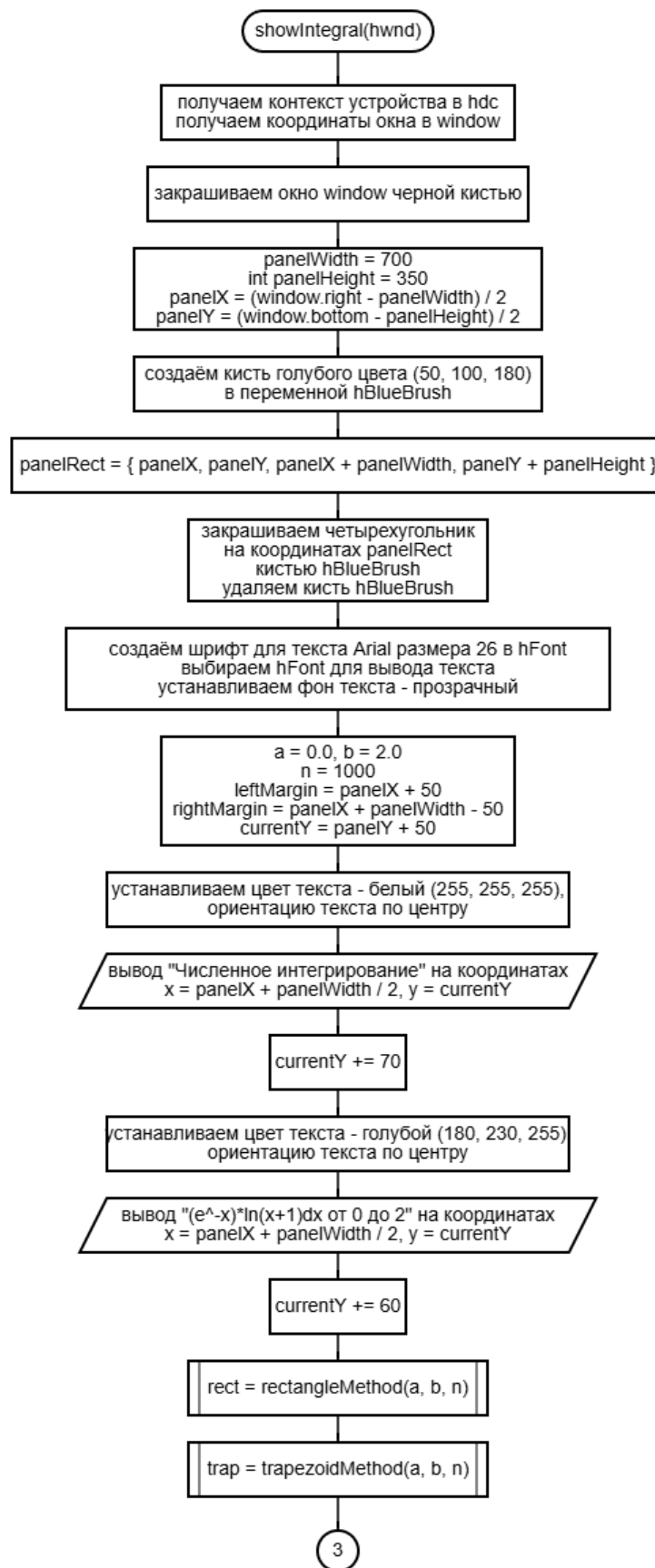
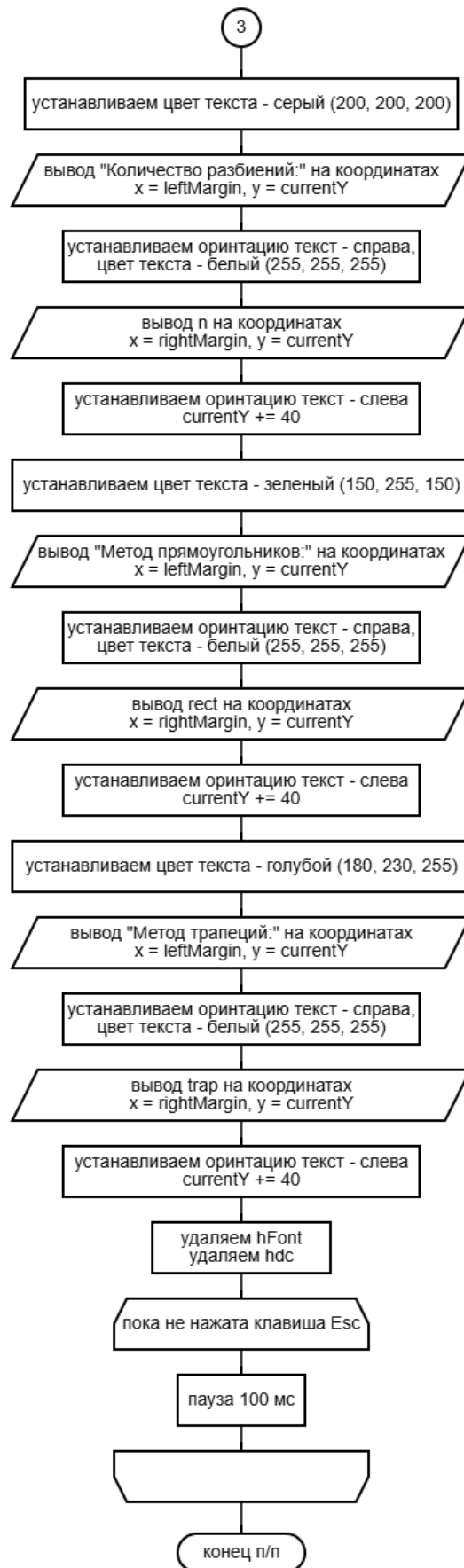


Рисунок 11 – схема алгоритма вывода результатов подсчётов интеграла.



Продолжение рисунка 11 – схема алгоритма вывода результатов подсчётов интеграла.

На рисунке 12 показан алгоритм подсчёта значения функций, которой задано уравнение

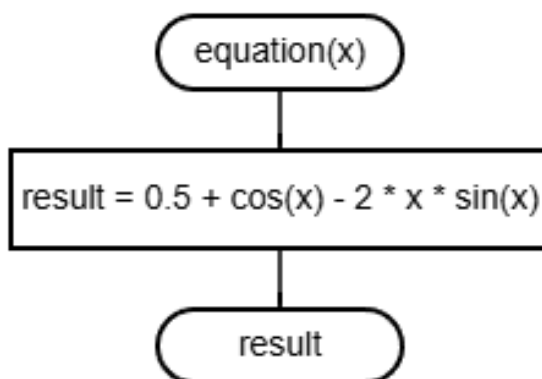


Рисунок 12 – схема алгоритма подсчёта значения функции из уравнения.

На рисунке 13 показана схема решения уравнения методом хорд.

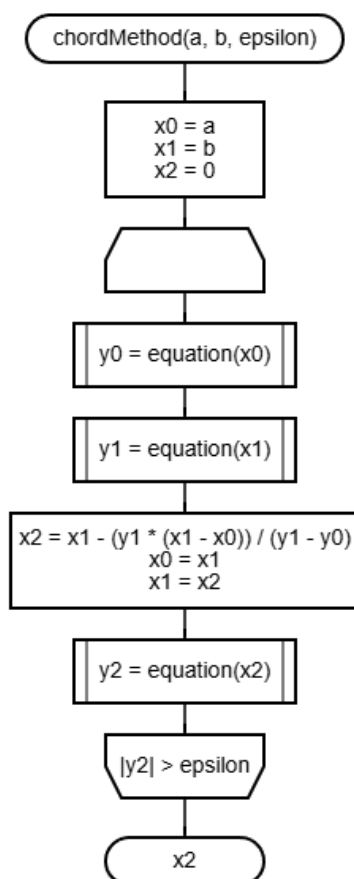


Рисунок 13 – схема алгоритма решения уравнения методом хорд.

На рисунке 14 показана схема решения уравнения методом бисекции.

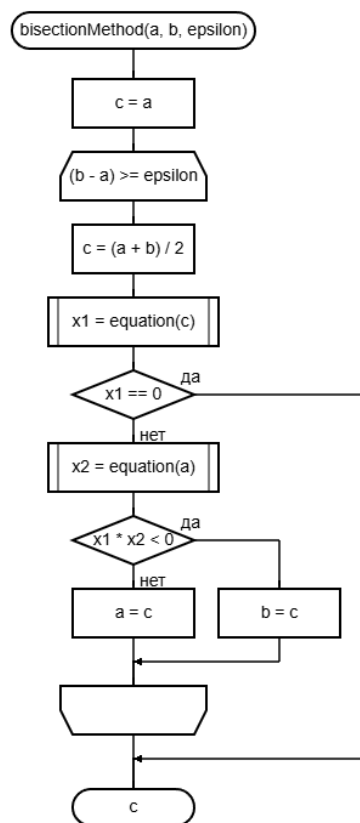


Рисунок 14 – схема алгоритма решения уравнения методом бисекции.

На рисунке 15 представлен алгоритм вывода решений уравнения методами хорд и бисекции.

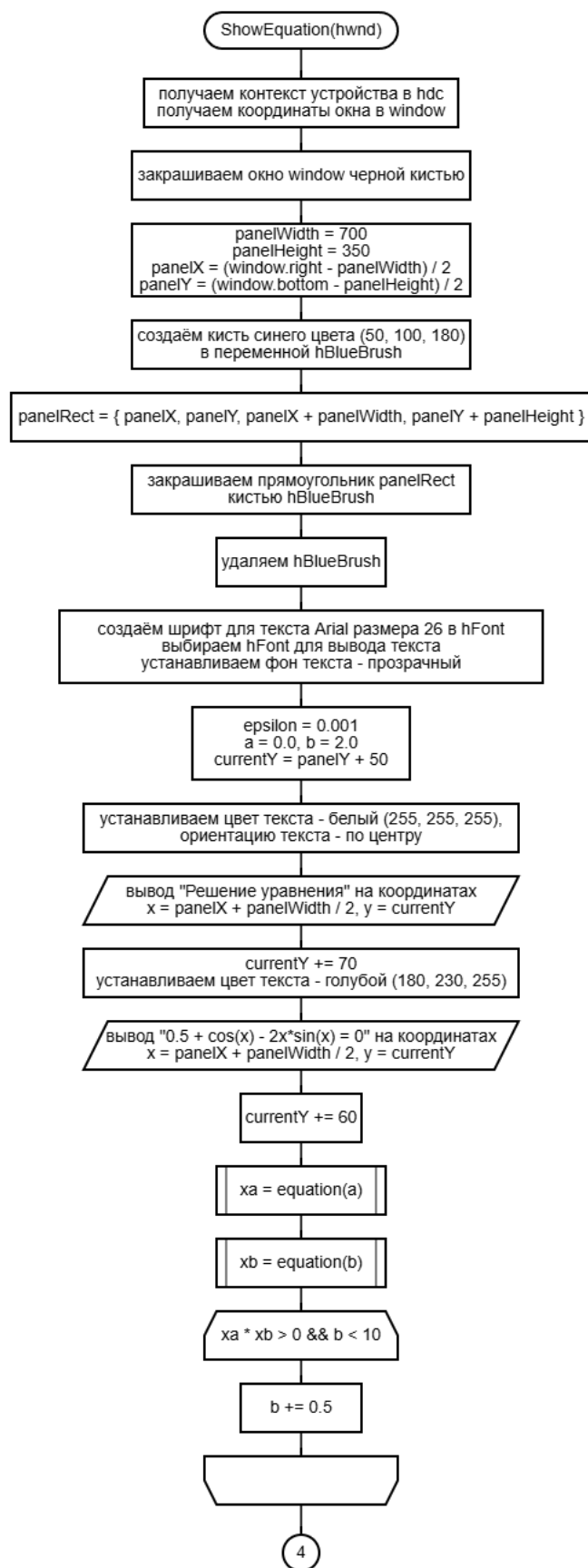
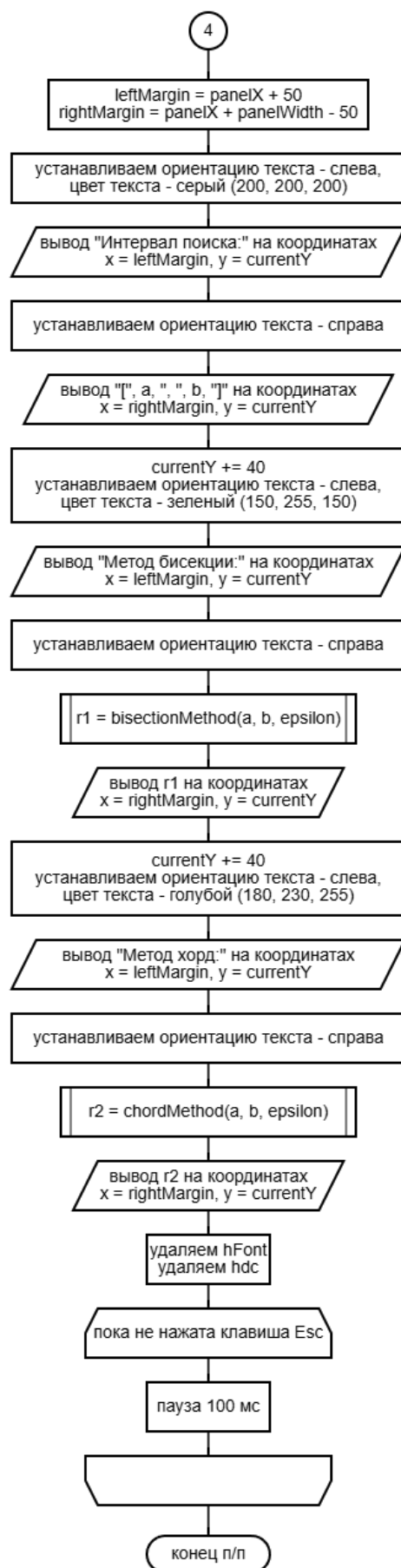


Рисунок 7 – схема алгоритма вывода решений уравнения.



Продолжение рисунка 15 – схема алгоритма вывода решений уравнения.

Код

functions.cpp

```
#include "functions.h"

const int MENU_ITEMS = 7;
const char* menuItems[MENU_ITEMS] = {
    "1. Заставка",
    "2. Об авторе",
    "3. Таблица",
    "4. График",
    "5. Интеграл",
    "6. Уравнение",
    "7. Выход"
};
```

functions.h

```
#pragma once
#define _USE_MATH_DEFINES
#include <windows.h>
#include <string>
#include <cstring>
#include <cmath>
#include <iomanip>
#include <sstream>
#include <vector>

using namespace std;

extern const int MENU_ITEMS;
extern const char* menuItems[];

void DrawMenu(HWND hwnd, int selected);
void HandleInput(HWND hwnd, int* selected);
void ShowAbout(HWND hwnd);
void ShowTable(HWND hwnd);
void ShowGraph(HWND hwnd);
void ShowIntegral(HWND hwnd);
void ShowEquation(HWND hwnd);
void ShowIntro(HWND hwnd);
```

main.cpp

```
#include "functions.h"

int main() {
    HWND hwnd = GetConsoleWindow();
    ShowWindow(hwnd, SW_MAXIMIZE);

    CONSOLE_CURSOR_INFO cci = { 1, FALSE };
    SetConsoleCursorInfo(GetStdHandle(STD_OUTPUT_HANDLE), &cci);
```



```

    int selected = 0;
    DrawMenu(hwnd, selected);

    while (true) {
        HandleInput(hwnd, &selected);
    }

    return 0;
}

```

menu.cpp

```
#include "functions.h"
```

```

void DrawMenu(HWND hwnd, int selected) {
    HDC hdc = GetDC(hwnd);
    RECT window;
    GetClientRect(hwnd, &window);

    FillRect(hdc, &window, (HBRUSH)GetStockObject(BLACK_BRUSH));

    const int menuWidth = 400;
    const int itemHeight = 40;
    const int menuHeight = MENU_ITEMS * itemHeight;

    const int menuLeft = (window.right - menuWidth) / 2;
    const int menuTop = (window.bottom - menuHeight) / 2;

    HFONT hNormalFont = CreateFont(28, 0, 0, 0, FW_NORMAL, FALSE,
FALSE, FALSE,
    DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
    CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
    DEFAULT_PITCH, L"Arial");
    HFONT hSelectedFont = CreateFont(28, 0, 0, 0, FW_NORMAL, TRUE,
FALSE, FALSE,
    DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
    CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
    DEFAULT_PITCH, L"Arial");

    SetTextAlign(hdc, TA_CENTER);
    SetTextColor(hdc, RGB(255, 255, 255));
    SetBkMode(hdc, TRANSPARENT);
    SelectObject(hdc, hNormalFont);
    TextOutA(hdc, window.right / 2, menuTop - 60, "Главное меню",
12);

    SetTextAlign(hdc, TA_LEFT);
    for (int i = 0; i < MENU_ITEMS; i++) {
        RECT itemRect = {
            menuLeft,
            menuTop + i * itemHeight,
            menuLeft + menuWidth,

```

```

        menuTop + (i + 1) * itemHeight
    };

    if (i == selected) {
        HBRUSH hBlueBrush = CreateSolidBrush(
255));
        FillRect(hdc, &itemRect, hBlueBrush);
        DeleteObject(hBlueBrush);

        SetTextColor(hdc, RGB(255, 255, 255));
        SelectObject(hdc, hSelectedFont);
    }
    else {
        SetTextColor(hdc, RGB(200, 200, 200));
        SelectObject(hdc, hNormalFont);
    }

    TextOutA(hdc, menuLeft + 10, menuTop + i * itemHeight + 8,
menuItems[i], (int)strlen(menuItems[i]));
}

DeleteObject(hNormalFont);
DeleteObject(hSelectedFont);
ReleaseDC(hwnd, hdc);
}

void HandleInput(HWND hwnd, int* selected) {
    if (GetAsyncKeyState(VK_UP) & 0x8000) {
        *selected = (*selected - 1 + MENU_ITEMS) % MENU_ITEMS;
        DrawMenu(hwnd, *selected);
        Sleep(150);
    }
    else if (GetAsyncKeyState(VK_DOWN) & 0x8000) {
        *selected = (*selected + 1) % MENU_ITEMS;
        DrawMenu(hwnd, *selected);
        Sleep(150);
    }
    else if (GetAsyncKeyState(VK_RETURN) & 0x8000) {
        if (*selected == MENU_ITEMS - 1) {
            HDC hdc = GetDC(hwnd);
            RECT window;
            GetClientRect(hwnd, &window);

            FillRect(hdc, &window,
(HBRUSH)GetStockObject(BLACK_BRUSH));

            SetTextColor(hdc, RGB(255, 255, 255));
            SetBkMode(hdc, TRANSPARENT);
            HFONT hFont = CreateFont(32, 0, 0, 0, FW_BOLD, FALSE,
FALSE, FALSE,
                DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
                CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,

```

```

        DEFAULT_PITCH, L"Arial");
    SelectObject(hdc, hFont);
    SetTextAlign(hdc, TA_CENTER);

    const char* msg = "Нажмите любую клавишу, чтобы закрыть
это окно...";
    TextOutA(hdc, window.right / 2, window.bottom / 2, msg,
(int)strlen(msg));

    DeleteObject(hFont);
    ReleaseDC(hwnd, hdc);
    exit(0);
}

switch (*selected) {
    case 0: ShowIntro(hwnd); break;
    case 1: ShowAbout(hwnd); break;
    case 2: ShowTable(hwnd); break;
    case 3: ShowGraph(hwnd); break;
    case 4: ShowIntegral(hwnd); break;
    case 5: ShowEquation(hwnd); break;
}

DrawMenu(hwnd, *selected);
}
}

```

intro.cpp

```
#include "functions.h"
```

```

void ShowIntro(HWND hwnd) {
    HDC hdc = GetDC(hwnd);
    RECT window;
    GetClientRect(hwnd, &window);

    const COLORREF colors[] = {
        RGB(255, 50, 50),    // красный
        RGB(50, 255, 50),    // зеленый
        RGB(50, 50, 255),    // синий
        RGB(255, 255, 50),    // желтый
        RGB(255, 50, 255),    // пурпурный
        RGB(50, 255, 255),    // голубой
        RGB(50, 200, 100),    // зеленый
        RGB(90, 90, 150),     // нежно-фиолетовый
        RGB(160, 10, 50),     // малиновый
        RGB(10, 100, 200)     // синий
    };

    float angle = 0.0f;
    float scale = 0.5f;
    float distanceMultiplier = 2.5f;
}

```

```

bool scaleIncreasing = true;
const float rotationSpeed = 0.02f;
const float scaleSpeed = 0.008f;

while (!(GetAsyncKeyState(VK_ESCAPE) & 0x8000)) {
    FillRect(hdc, &window, (HBRUSH)GetStockObject(BLACK_BRUSH));

    const int centerX = window.right / 2;
    const int centerY = window.bottom / 2;

    angle += rotationSpeed;
    if (scaleIncreasing) {
        scale += scaleSpeed;
        if (scale > 1.5f) scaleIncreasing = false;
    }
    else {
        scale -= scaleSpeed;
        if (scale < 0.4f) scaleIncreasing = true;
    }

    for (int i = 0; i < 10; i++) {
        float currentAngle = angle + i * (2 * M_PI / 6);
        int distance = (100 + i * 30) * scale *
distanceMultiplier;

        int x = centerX + distance * 1.5f * cos(currentAngle);
        int y = centerY + distance * 0.8f * sin(currentAngle);

        int size = 40 + 30 * sin(angle * 2 + i);

        HBRUSH hBrush = CreateSolidBrush(colors[i]);
        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);

        if (i % 2 == 0) {
            Ellipse(hdc, x - size, y - size, x + size, y +
size);
        }
        else {
            POINT diamond[4] = {
                {x, y - size},
                {x + size, y},
                {x, y + size},
                {x - size, y}
            };
            Polygon(hdc, diamond, 4);
        }

        SelectObject(hdc, hOldBrush);
        DeleteObject(hBrush);
    }

    Sleep(16);
}

```

```

        GdiFlush();
    }

    ReleaseDC(hwnd, hdc);
}

```

about.cpp

```
#include "functions.h"
```

```

void ShowAbout(HWND hwnd) {
    HDC hdc = GetDC(hwnd);

    RECT window;
    GetClientRect(hwnd, &window);
    FillRect(hdc, &window, (HBRUSH)GetStockObject(BLACK_BRUSH));

    HFONT hFont = CreateFont(28, 0, 0, 0, FW_BOLD, FALSE, FALSE,
FALSE,
        DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
        CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
        DEFAULT_PITCH, L"Arial");
    SelectObject(hdc, hFont);

    SetTextColor(hdc, RGB(255, 255, 255));
    SetTextAlign(hdc, TA_CENTER);
    SetBkMode(hdc, TRANSPARENT);

    TextOutA(hdc, window.right / 2, window.bottom / 2 - 150,
"Информация об авторе", 20);

    HFONT hTextFont = CreateFont(24, 0, 0, 0, FW_NORMAL, FALSE,
FALSE, FALSE,
        DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
        CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
        DEFAULT_PITCH, L"Arial");
    SelectObject(hdc, hTextFont);

    TextOutA(hdc, window.right / 2, window.bottom / 2 - 80, "ФИО:
Шмидт Антон Владиславович", 30);
    TextOutA(hdc, window.right / 2, window.bottom / 2 - 40, "Дата
рождения: 17.10.2006", 25);
    TextOutA(hdc, window.right / 2, window.bottom / 2, "Факультет:
ФИТиКС", 17);
    TextOutA(hdc, window.right / 2, window.bottom / 2 + 40, "Группа:
ИБТ-244", 15);

    DeleteObject(hFont);
    DeleteObject(hTextFont);
    ReleaseDC(hwnd, hdc);
}

```

```

    while (!(GetAsyncKeyState(VK_ESCAPE) & 0x8000)) Sleep(100);
}

```

table.cpp

```

#include "functions.h"

```

```

void ShowTable(HWND hwnd) {
    HDC hdc = GetDC(hwnd);
    RECT window;
    GetClientRect(hwnd, &window);

    FillRect(hdc, &window, (HBRUSH)GetStockObject(BLACK_BRUSH));

    HFONT hFont = CreateFont(24, 0, 0, 0, FW_NORMAL, FALSE, FALSE,
FALSE,
        DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
        CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
        DEFAULT_PITCH, L"Courier New");
    SelectObject(hdc, hFont);
    SetBkMode(hdc, TRANSPARENT);

    SetTextColor(hdc, RGB(255, 255, 255));
    TextOutA(hdc, window.right / 2 - 300, 50, "Таблица значений
функций:", 24);
    TextOutA(hdc, window.right / 2 - 300, 80, "y1 = 5 - 3*cos(x)",
17);
    TextOutA(hdc, window.right / 2 - 300, 110, "y2 = sqrt(1 +
sin(x)^2)", 23);
    TextOutA(hdc, window.right / 2 - 300, 140, "Интервал: [0, 2Pi],
шагов: 20", 30);

    const int rows = 21;
    const int cols = 4;
    const int cellWidth = 150;
    const int cellHeight = 30;
    const int tableWidth = cols * cellWidth;
    const int tableHeight = rows * cellHeight;
    const int tableX = (window.right - tableWidth) / 2;
    const int tableY = 180;

    struct TableData {
        int step;
        double x;
        double y1;
        double y2;
    } data[20];

    double minY1 = 1000, maxY1 = -1000;
    double minY2 = 1000, maxY2 = -1000;

    for (int i = 0; i < 20; i++) {

```

```

data[i].step = i + 1;
data[i].x = (2 * M_PI * i) / 19.0;
data[i].y1 = 5 - 3 * cos(data[i].x);
data[i].y2 = sqrt(1 + pow(sin(data[i].x), 2));

if (data[i].y1 < minY1) minY1 = data[i].y1;
if (data[i].y1 > maxY1) maxY1 = data[i].y1;
if (data[i].y2 < minY2) minY2 = data[i].y2;
if (data[i].y2 > maxY2) maxY2 = data[i].y2;
}

HPEN hWhitePen = CreatePen(PS_SOLID, 1, RGB(255, 255, 255));
HPEN hOldPen = (HPEN)SelectObject(hdc, hWhitePen);

for (int row = 0; row < rows; row++) {
    for (int col = 0; col < cols; col++) {
        RECT cell = {
            tableX + col * cellWidth,
            tableY + row * cellHeight,
            tableX + (col + 1) * cellWidth,
            tableY + (row + 1) * cellHeight
        };

        MoveToEx(hdc, cell.left, cell.top, NULL);
        LineTo(hdc, cell.right, cell.top);
        LineTo(hdc, cell.right, cell.bottom);
        LineTo(hdc, cell.left, cell.bottom);
        LineTo(hdc, cell.left, cell.top);

        if (row == 0) {
            SetTextColor(hdc, RGB(255, 255, 255));
            const char* headers[] = { "Var", "X", "y1", "y2" };
            TextOutA(hdc, cell.left + 10, cell.top + 5,
headers[col], (int)strlen(headers[col]));
        }
        else {
            TableData& item = data[row - 1];
            ostringstream oss;
            oss << fixed << setprecision(4);

            switch (col) {
                case 0: oss << item.step; break;
                case 1: oss << item.x; break;
                case 2:
                    oss << item.y1;
                    if (item.y1 == minY1) {
                        SetTextColor(hdc, RGB(0, 0, 255));
                    }
                    else if (item.y1 == maxY1) {
                        SetTextColor(hdc, RGB(255, 0, 0));
                    }
                    else {

```

```

        SetTextColor(hdc, RGB(255, 255, 255));
    }
    break;
case 3:
    oss << item.y2;
    if (item.y2 == minY2) {
        SetTextColor(hdc, RGB(0, 0, 255));
    }
    else if (item.y2 == maxY2) {
        SetTextColor(hdc, RGB(255, 0, 0));
    }
    else {
        SetTextColor(hdc, RGB(255, 255, 255));
    }
    break;
}

    string text = oss.str();
    TextOutA(hdc, cell.left + 10, cell.top + 5,
text.c_str(), (int)text.length());
    SetTextColor(hdc, RGB(255, 255, 255));
}
}

SelectObject(hdc, hOldPen);
DeleteObject(hWhitePen);

SetTextColor(hdc, RGB(255, 0, 0));
TextOutA(hdc, tableX, tableY + rows * cellHeight + 20, "Красный
- максимум", 18);
SetTextColor(hdc, RGB(0, 0, 255));
TextOutA(hdc, tableX, tableY + rows * cellHeight + 50, "Синий -
минимум", 15);

DeleteObject(hFont);
ReleaseDC(hwnd, hdc);

while (!(GetAsyncKeyState(VK_ESCAPE) & 0x8000)) Sleep(100);
}

```

graph.cpp

```
#include "functions.h"
```

```

void ShowGraph(HWND hwnd) {
    HDC hdc = GetDC(hwnd);
    RECT window;
    GetClientRect(hwnd, &window);

    FillRect(hdc, &window, (HBRUSH)GetStockObject(BLACK_BRUSH));
}

```



```

HFONT hFont = CreateFont(20, 0, 0, 0, FW_NORMAL, FALSE, FALSE,
FALSE,
    DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
    CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
    DEFAULT_PITCH, L"Arial");
SelectObject(hdc, hFont);
SetBkMode(hdc, TRANSPARENT);

const double xMin = 0;
const double xMax = 2 * M_PI;
const double yMin = 0;
const double yMax = 8;

const int margin = 50;
const int graphWidth = window.right - 2 * margin;
const int graphHeight = window.bottom - 2 * margin;
const int originX = margin;
const int originY = window.bottom - margin;

// подписи
SetTextColor(hdc, RGB(255, 255, 255));
TextOutA(hdc, originX + graphWidth + 20, originY - 10, "x", 1);
TextOutA(hdc, originX - 5, originY - graphHeight - 40, "y", 1);

HPEN hGridPen = CreatePen(PS_DOT, 1, RGB(100, 100, 100));
SelectObject(hdc, hGridPen);

const char* xLabels[] = { "0", "Pi/2", "Pi", "3Pi/2", "2Pi" };
double xValues[] = { 0, M_PI / 2, M_PI, 3 * M_PI / 2, 2 * M_PI
};

// ось X
for (int i = 0; i < 5; i++) {
    // вертикальные линии сетки
    int xPos = originX + (xValues[i] / (2 * M_PI)) * graphWidth;
    MoveToEx(hdc, xPos, originY, NULL);
    LineTo(hdc, xPos, originY - graphHeight);

    TextOutA(hdc, xPos - 15, originY + 5, xLabels[i],
(int)strlen(xLabels[i]));
}

// ось Y
for (int i = 1; i <= 8; i++) {
    // горизонтальные линии сетки
    int yPos = originY - (i / 8.0) * graphHeight;
    MoveToEx(hdc, originX, yPos, NULL);
    LineTo(hdc, originX + graphWidth, yPos);

    ostringstream oss;
    oss << i;
    string label = oss.str();

```

```

        TextOutA(hdc, originX - 20, yPos - 10, label.c_str(),
(int)label.length());
    }

    const int points = 500;
    POINT y1Points[points], y2Points[points];

    for (int i = 0; i < points; i++) {
        double x = xMin + (xMax - xMin) * i / (points - 1);
        double y1 = 5 - 3 * cos(x);
        double y2 = sqrt(1 + pow(sin(x), 2));

        y1Points[i].x = originX + (x / (2 * M_PI)) * graphWidth;
        y1Points[i].y = originY - ((y1 - yMin) / (yMax - yMin)) *
graphHeight;

        y2Points[i].x = originX + (x / (2 * M_PI)) * graphWidth;
        y2Points[i].y = originY - ((y2 - yMin) / (yMax - yMin)) *
graphHeight;
    }

    HPEN hAxisPen = CreatePen(PS_SOLID, 2, RGB(255, 255, 255));
    SelectObject(hdc, hAxisPen);

    // ось X
    MoveToEx(hdc, originX, originY, NULL);
    LineTo(hdc, originX + graphWidth, originY);
    // ось Y
    MoveToEx(hdc, originX, originY, NULL);
    LineTo(hdc, originX, originY - graphHeight);

    // стрелка ось X
    MoveToEx(hdc, originX + graphWidth, originY, NULL);
    LineTo(hdc, originX + graphWidth - 10, originY - 5);
    MoveToEx(hdc, originX + graphWidth, originY, NULL);
    LineTo(hdc, originX + graphWidth - 10, originY + 5);
    // стрелка ось Y
    MoveToEx(hdc, originX, originY - graphHeight, NULL);
    LineTo(hdc, originX - 5, originY - graphHeight + 10);
    MoveToEx(hdc, originX, originY - graphHeight, NULL);
    LineTo(hdc, originX + 5, originY - graphHeight + 10);

    // график y1 красный
    HPEN hY1Pen = CreatePen(PS_SOLID, 2, RGB(255, 0, 0));
    SelectObject(hdc, hY1Pen);
    Polyline(hdc, y1Points, points);

    // график y2 синий
    HPEN hY2Pen = CreatePen(PS_SOLID, 2, RGB(0, 0, 255));
    SelectObject(hdc, hY2Pen);
    Polyline(hdc, y2Points, points);

```

```

// подписи графиков
SetTextColor(hdc, RGB(255, 0, 0));
TextOutA(hdc, originX + 100, originY - graphHeight + 30, "y1 = 5
- 3*cos(x)", 17);

SetTextColor(hdc, RGB(0, 0, 255));
TextOutA(hdc, originX + 100, originY - graphHeight + 60, "y2 =
sqrt(1 + sin(x)^2)", 23);

DeleteObject(hFont);
DeleteObject(hAxisPen);
DeleteObject(hGridPen);
DeleteObject(hY1Pen);
DeleteObject(hY2Pen);
ReleaseDC(hwnd, hdc);

while (!(GetAsyncKeyState(VK_ESCAPE) & 0x8000)) Sleep(100);
}

```

integral.cpp

```
#include "functions.h"
```

```
double f(double x) { return exp(-x) * log(x + 1); }
```

```
double rectangleMethod(double a, double b, int n) {
    double h = (b - a) / n;
    double sum = 0.0;
    for (int i = 0; i < n; i++) sum += f(a + (i + 0.5) * h);
    return sum * h;
}

```

```
double trapezoidMethod(double a, double b, int n) {
    double h = (b - a) / n;
    double sum = (f(a) + f(b)) / 2.0;
    for (int i = 1; i < n; i++) sum += f(a + i * h);
    return sum * h;
}

```

```
void ShowIntegral(HWND hwnd) {
    HDC hdc = GetDC(hwnd);
    RECT window;
    GetClientRect(hwnd, &window);
    FillRect(hdc, &window, (HBRUSH)GetStockObject(BLACK_BRUSH));

    // фон
    const int panelWidth = 700;
    const int panelHeight = 350;
    const int panelX = (window.right - panelWidth) / 2;
    const int panelY = (window.bottom - panelHeight) / 2;

    HBRUSH hBlueBrush = CreateSolidBrush(RGB(50, 100, 180));
}

```

```

    RECT panelRect = { panelX, panelY, panelX + panelWidth, panelY +
panelHeight };
    FillRect(hdc, &panelRect, hBlueBrush);
    DeleteObject(hBlueBrush);

    HFONT hFont = CreateFont(26, 0, 0, 0, FW_NORMAL, FALSE, FALSE,
FALSE,
    DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
    CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
    DEFAULT_PITCH, L"Arial");
    SelectObject(hdc, hFont);
    SetBkMode(hdc, TRANSPARENT);

    const double a = 0.0, b = 2.0;
    const int n = 1000;
    const int leftMargin = panelX + 50;
    const int rightMargin = panelX + panelWidth - 50;
    int currentY = panelY + 50;

    SetTextColor(hdc, RGB(255, 255, 255));
    SetTextAlign(hdc, TA_CENTER);
    TextOutA(hdc, panelX + panelWidth / 2, currentY, "Численное
интегрирование", 24);
    currentY += 70;

    SetTextColor(hdc, RGB(180, 230, 255));
    TextOutA(hdc, panelX + panelWidth / 2, currentY, "(e^–
x)*ln(x+1)dx от 0 до 2", 26);
    SetTextAlign(hdc, TA_LEFT);
    currentY += 60;

    auto DrawRow = [&](const char* label, const string& value,
COLORREF color) {
        SetTextColor(hdc, color);
        TextOutA(hdc, leftMargin, currentY, label,
(int)strlen(label));

        SetTextAlign(hdc, TA_RIGHT);
        SetTextColor(hdc, RGB(255, 255, 255));
        TextOutA(hdc, rightMargin, currentY, value.c_str(),
(int)value.length());
        SetTextAlign(hdc, TA_LEFT);

        currentY += 40;
    };

    double rect = rectangleMethod(a, b, n);
    double trap = trapezoidMethod(a, b, n);

    ostringstream oss;
    oss << n;
    DrawRow("Количество разбиений:", oss.str(), RGB(200, 200, 200));

```

```

    oss.str("");
    oss << fixed << setprecision(6) << rect;
    DrawRow("Метод прямоугольников:", oss.str(), RGB(150, 255,
150));

    oss.str("");
    oss << fixed << setprecision(6) << trap;
    DrawRow("Метод трапеций:", oss.str(), RGB(180, 230, 255));

    DeleteObject(hFont);
    ReleaseDC(hwnd, hdc);

    while (!(GetAsyncKeyState(VK_ESCAPE) & 0x8000)) Sleep(100);
}

```

equation.cpp

```
#include "functions.h"
```

```

static double equation(double x) {
    return 0.5 + cos(x) - 2 * x * sin(x);
}

static double bisectionMethod(double a, double b, double epsilon) {
    double c = a;

    while ((b - a) >= epsilon) {
        c = (a + b) / 2;
        if (equation(c) == 0.0) break;
        else if (equation(c) * equation(a) < 0) b = c;
        else a = c;
    }

    return c;
}

static double chordMethod(double a, double b, double epsilon) {
    double x0 = a;
    double x1 = b;
    double x2 = 0;

    do {
        x2 = x1 - (equation(x1) * (x1 - x0)) / (equation(x1) -
equation(x0));
        x0 = x1;
        x1 = x2;
    } while (fabs(equation(x2)) > epsilon);

    return x2;
}

```

```

void ShowEquation(HWND hwnd) {
    HDC hdc = GetDC(hwnd);
    RECT window;
    GetClientRect(hwnd, &window);
    FillRect(hdc, &window, (HBRUSH)GetStockObject(BLACK_BRUSH));

    const int panelWidth = 700;
    const int panelHeight = 350;
    const int panelX = (window.right - panelWidth) / 2;
    const int panelY = (window.bottom - panelHeight) / 2;

    HBRUSH hBlueBrush = CreateSolidBrush(RGB(50, 100, 180));
    RECT panelRect = { panelX, panelY, panelX + panelWidth, panelY +
panelHeight };
    FillRect(hdc, &panelRect, hBlueBrush);
    DeleteObject(hBlueBrush);

    HFONT hFont = CreateFont(26, 0, 0, 0, FW_NORMAL, FALSE, FALSE,
FALSE,
    DEFAULT_CHARSET, OUT_DEFAULT_PRECIS,
    CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
    DEFAULT_PITCH, L"Arial");
    SelectObject(hdc, hFont);
    SetBkMode(hdc, TRANSPARENT);

    const double epsilon = 0.001;
    double a = 0.0, b = 2.0;
    int currentY = panelY + 50;

    SetTextColor(hdc, RGB(255, 255, 255));
    SetTextAlign(hdc, TA_CENTER);
    TextOutA(hdc, panelX + panelWidth / 2, currentY, "Решение
уравнения", 17);
    currentY += 70;

    SetTextColor(hdc, RGB(180, 230, 255));
    TextOutA(hdc, panelX + panelWidth / 2, currentY, "0.5 + cos(x) -
2x*sin(x) = 0", 29);
    currentY += 60;

    while (equation(a) * equation(b) > 0 && b < 10.0) b += 0.5;

    const int leftMargin = panelX + 50;
    const int rightMargin = panelX + panelWidth - 50;

    SetTextAlign(hdc, TA_LEFT);
    SetTextColor(hdc, RGB(200, 200, 200));
    TextOutA(hdc, leftMargin, currentY, "Интервал поиска:", 16);

    SetTextAlign(hdc, TA_RIGHT);
    ostringstream oss;
    oss << "[" << fixed << setprecision(1) << a << ", " << b << "]";

```

```

    TextOutA(hdc, rightMargin, currentY, oss.str().c_str(),
(int)oss.str().length());
    currentY += 40;

    SetTextAlign(hdc, TA_LEFT);
    SetTextColor(hdc, RGB(150, 255, 150));
    TextOutA(hdc, leftMargin, currentY, "Метод бисекции:", 15);

    SetTextAlign(hdc, TA_RIGHT);
    oss.str("");
    oss << fixed << setprecision(5) << bisectionMethod(a, b,
epsilon);
    TextOutA(hdc, rightMargin, currentY, oss.str().c_str(),
(int)oss.str().length());
    currentY += 40;

    SetTextAlign(hdc, TA_LEFT);
    SetTextColor(hdc, RGB(180, 230, 255));
    TextOutA(hdc, leftMargin, currentY, "Метод хорд:", 11);

    SetTextAlign(hdc, TA_RIGHT);
    oss.str("");
    oss << fixed << setprecision(5) << chordMethod(a, b, epsilon);
    TextOutA(hdc, rightMargin, currentY, oss.str().c_str(),
(int)oss.str().length());

    DeleteObject(hFont);
    ReleaseDC(hwnd, hdc);

    while (!(GetAsyncKeyState(VK_ESCAPE) & 0x8000)) Sleep(100);
}

```

Разработка пользовательского интерфейса

На следующих рисунках представлен интерфейс всех состояний программы.

Рисунок 16 показывает главное меню программы, из которого производится запуск всех подпрограмм.

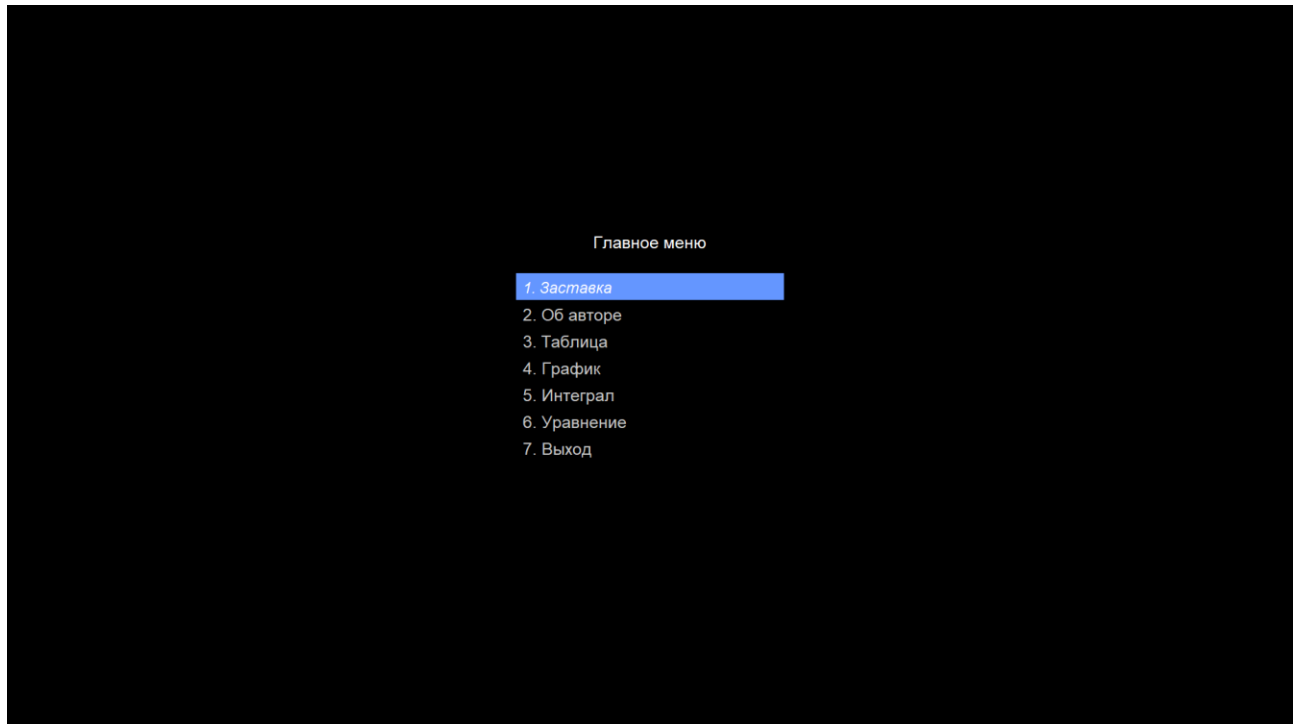


Рисунок 16 – интерфейс главного меню.

На рисунке 17 показан кадр из анимированной заставки программы.

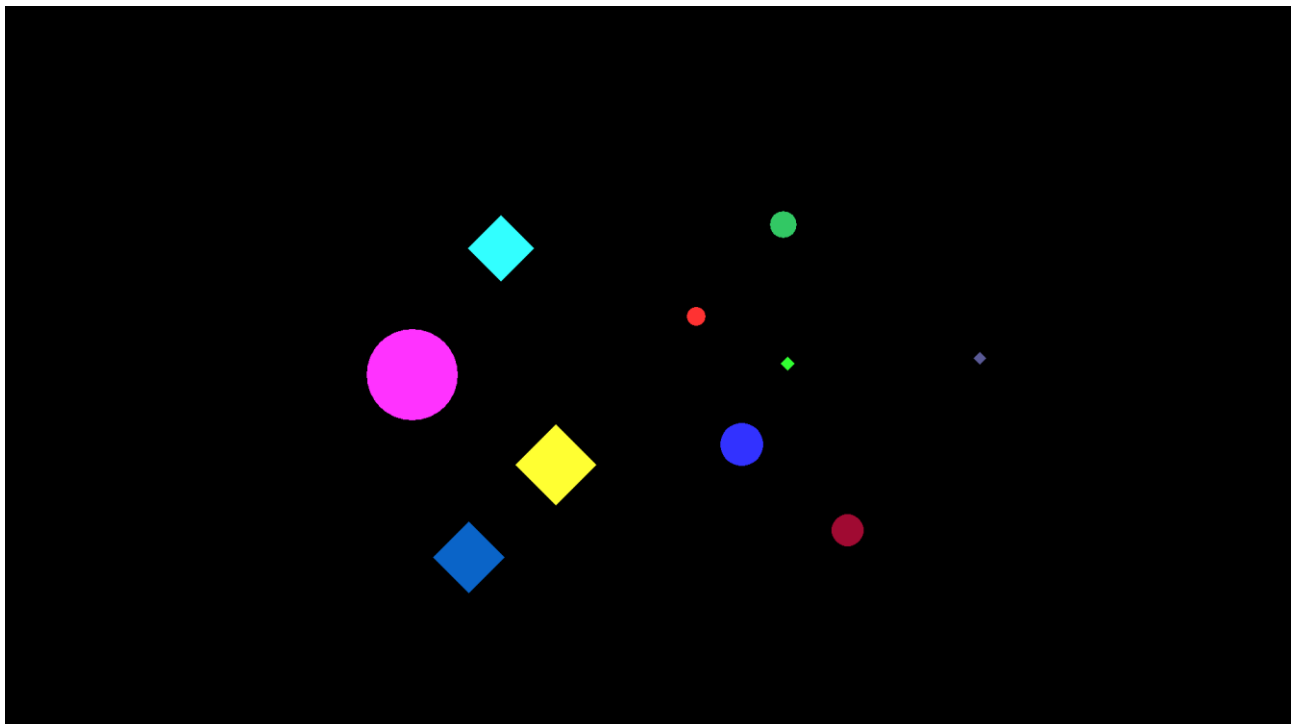


Рисунок 17 – динамическая заставка.

На рисунке 18 показан вывод информации об авторе программы, а именно: ФИО, даты рождения, факультета и группы.

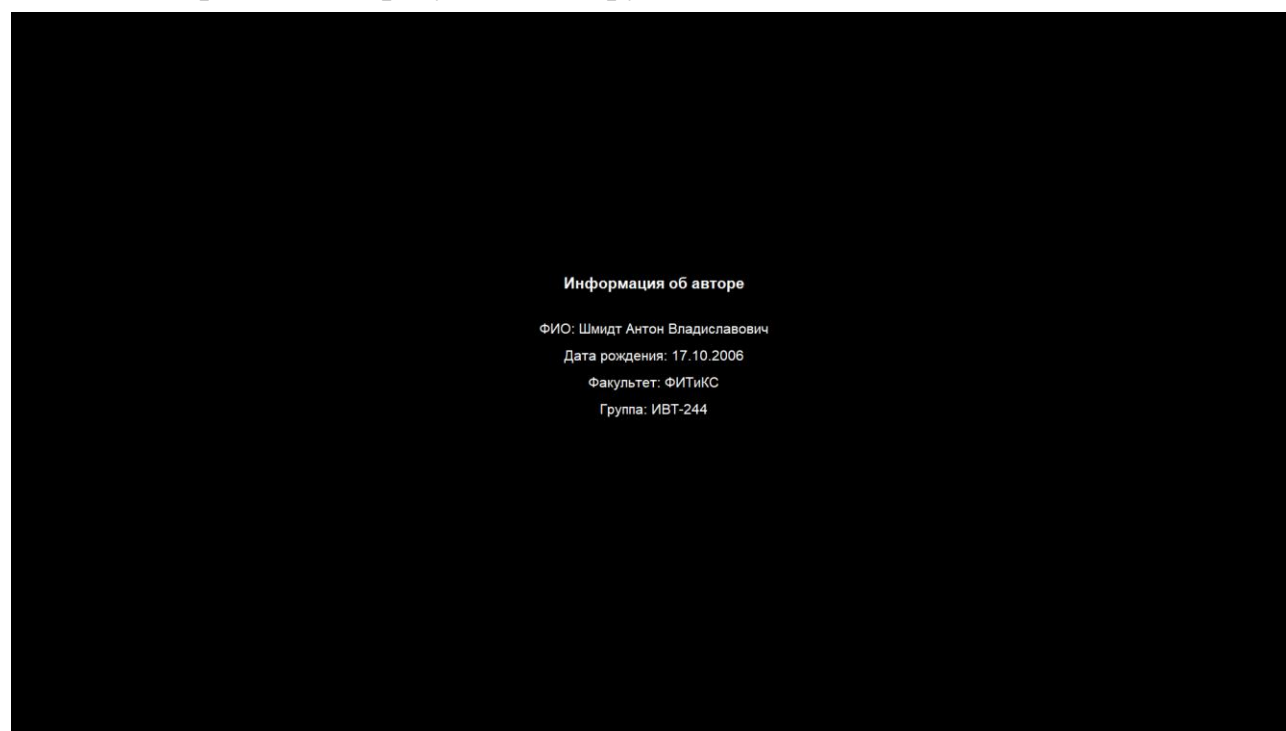


Рисунок 18 – вывод информации об авторе.

На рисунке 19 показана отрисовка таблицы с табуляцией функций, выделением минимальных и максимальных значений.

Таблица значений функций
 $y1 = 5 - 3 \cdot \cos(x)$
 $y2 = \sqrt{1 + \sin(x)^2}$
Интервал: $[0, 2\pi]$, шагов: 20

Шаг	X	y1	y2
1	0.0000	2.0000	1.0000
2	0.3307	2.1625	1.0514
3	0.6614	2.6326	1.1736
4	0.9921	3.3592	1.3042
5	1.3228	4.2635	1.3927
6	1.6535	5.2477	1.4118
7	1.9842	6.2051	1.3560
8	2.3149	7.0318	1.2415
9	2.6456	7.6384	1.1075
10	2.9762	7.9591	1.0135
11	3.3069	7.9591	1.0135
12	3.6376	7.6384	1.1075
13	3.9683	7.0318	1.2415
14	4.2990	6.2051	1.3560
15	4.6297	5.2477	1.4118
16	4.9604	4.2635	1.3927
17	5.2911	3.3592	1.3042
18	5.6218	2.6326	1.1736
19	5.9525	2.1625	1.0514
20	6.2832	2.0000	1.0000

Красный – максимум
Синий – минимум

Рисунок 19 – вывод таблицы табуляции функций.

На рисунке 20 показана отрисовка графиков двух функций на координатной плоскости.

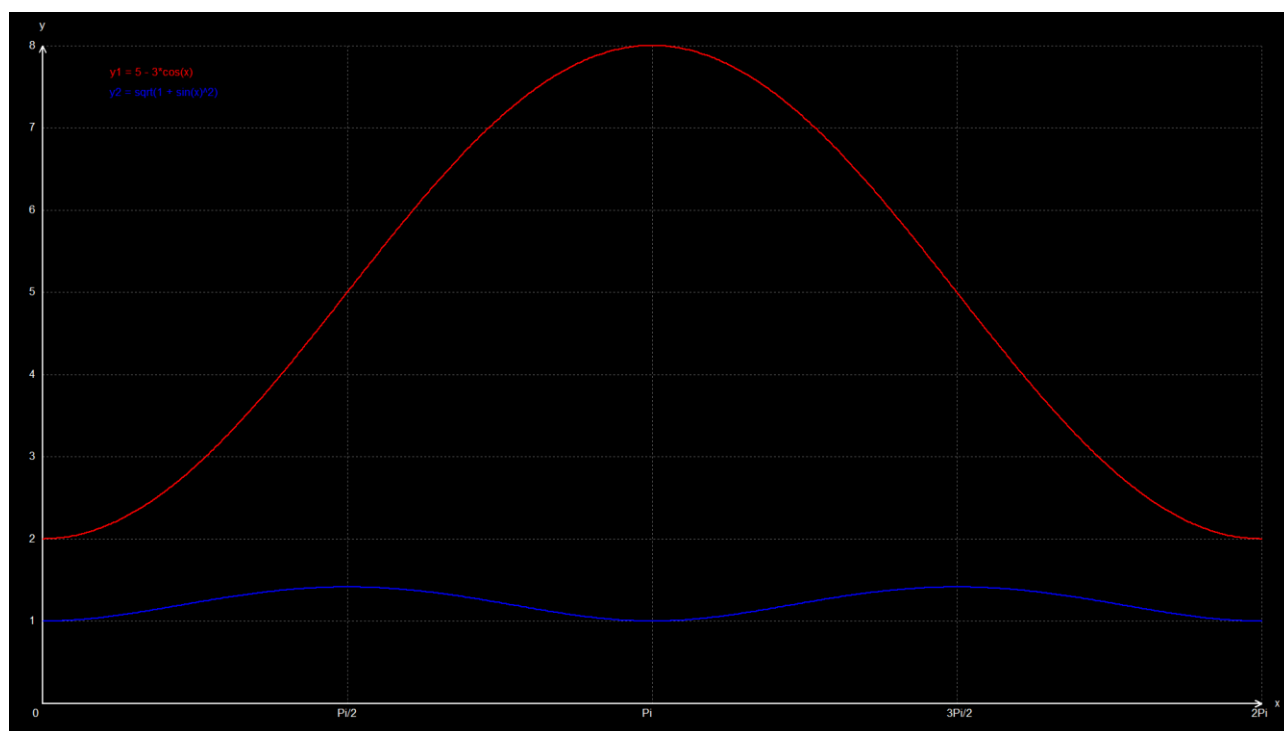


Рисунок 20 – вывод графиков функций.

На рисунке 21 показан вывод решений определенного интеграла методами прямоугольников и трапеций.



Рисунок 21 – вывод решений интеграла.

На рисунке 22 показан вывод решений уравнения методами бисекции и хорд.



Рисунок 22 – вывод решений уравнения.

На рисунке 23 показан вывод после того, как пользователь выбрал пункт «7. Выход» в меню и нажал Enter.

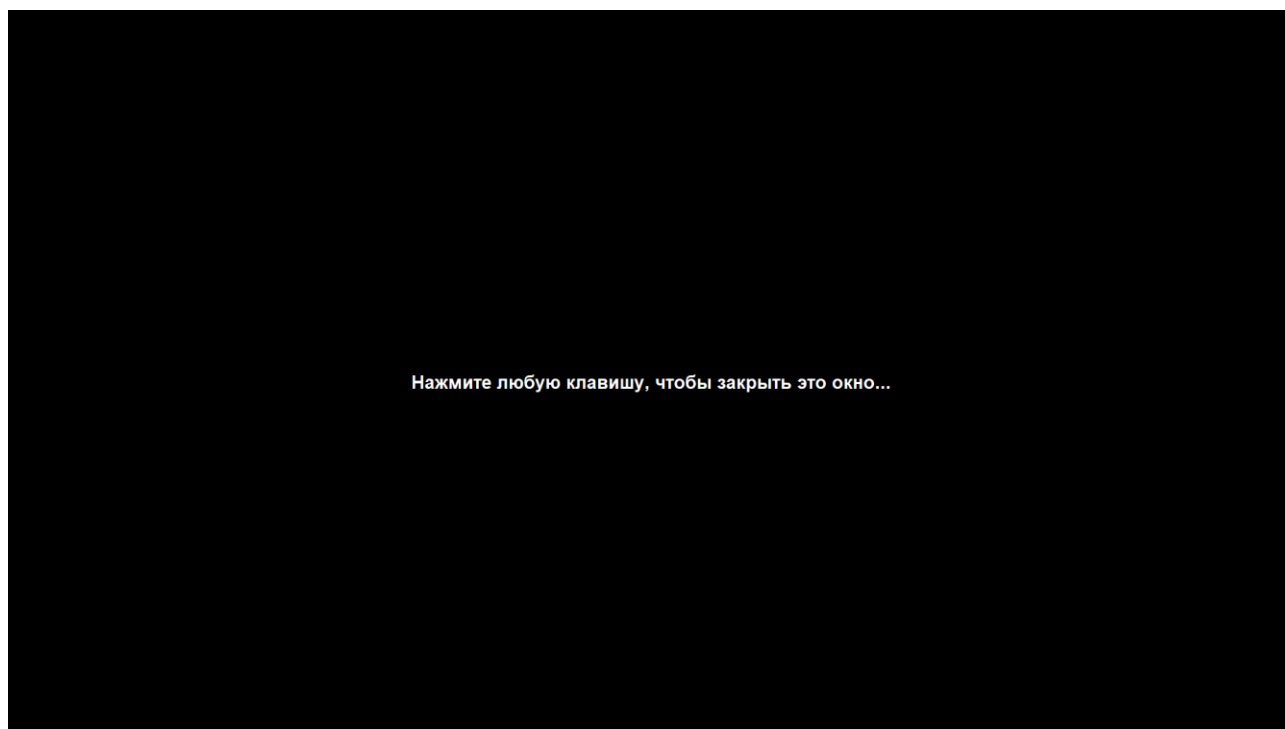


Рисунок 8 – вывод финального состояния при выходе.

Сравнивая вывод программы на рисунке 20 с полученными в Excel графиками функций в таблице 1 (где Ряд1 – функция $y_1 = 5 - 3 \times \cos(x)$ и Ряд2 – функция $y_2 = \sqrt[2]{1 + \sin(x)^2}$), мы можем убедиться, что наша программа работает исправно.

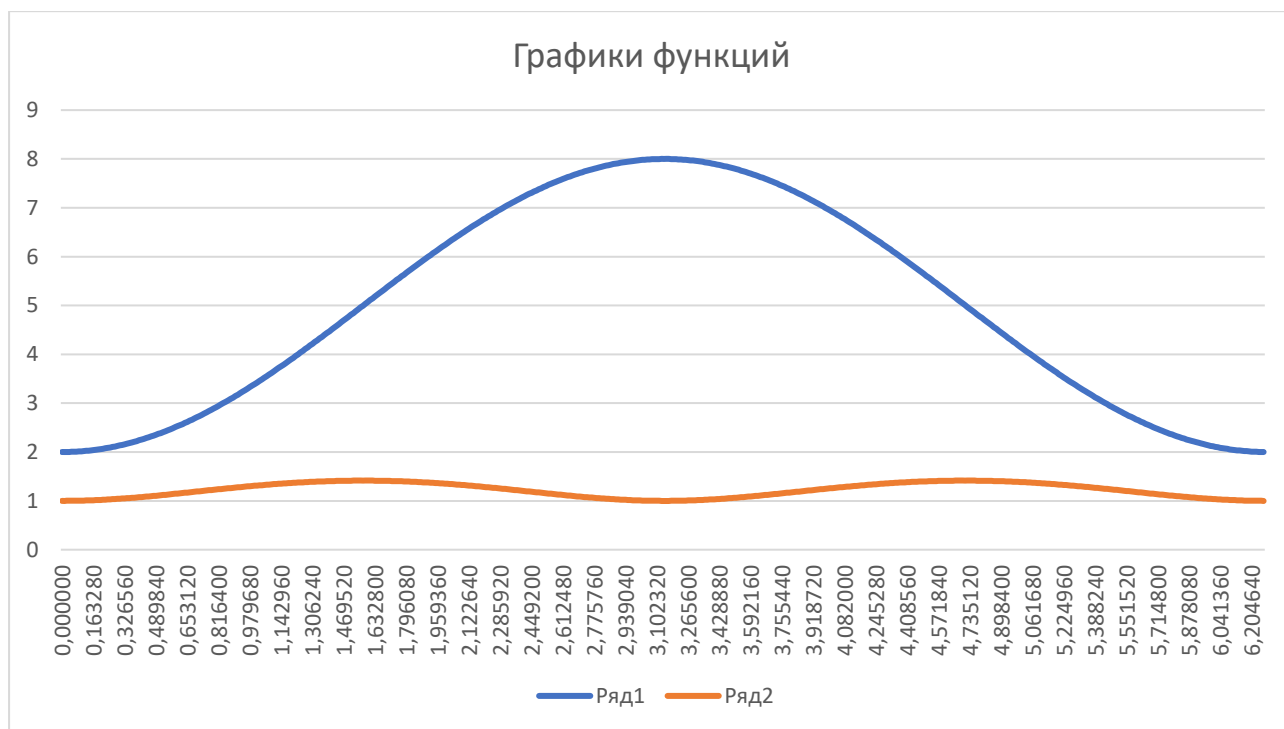


Таблица 1 – графики функций.

Список литературы

1. Березин, Б.И. Начальный курс Си и С++ / Б.И. Березин, С.И. Березин. – М.: Диалог-МИФИ, 1996. – 288 с.
2. ГОСТ 19.701-90. ЕСПД. Схемы алгоритмов, программ, данных и систем. – М.: Изд-во стандартов, 1991. – 26 с.
3. Крячков, А.В. и др. Программирование на С и С++. Практикум. / А.В. Крячков и др. – М.: Радио и связь, 1997. – 344 с.
4. Макогон, В.С. Язык программирования Си для начинающих / В.С. Макогон. – Одесса: НПФ "АСТРОПРИНТ", 1993. – 96 с.
5. Подбельский, В.В. Программирование на языке Си / В.В. Подбельский, С.С. Фомин. – М.: 2000 – 600 с.
6. Флоренсов, А.Н. Введение в программирование. Семантический подход: учеб. пособие / А.Н. Флоренсов. – Омск: Изд-во ОмГТУ, 1998. – 220 с.
7. Шафеева, О.П. Технологии программирования. С++: учеб. пособие / О.П. Шафеева. – Омск: Изд-во ОмГТУ, 2007. – 80 с.
8. Шилл, Г. Справочник программиста по С/С++. Учеб. Пособие / Г. Шилл – М.: Издательский дом «Вильямс», 2000. – 448 с.